COMPLETE_SECURITY_GUIDE

HelixCode Security Guide

Overview

HelixCode implements a comprehensive, zero-tolerance security policy designed for enterprise-grade distributed AI development platforms. This guide covers all security aspects including authentication, authorization, input validation, network security, and automated security scanning.

Security Principles

Zero-Tolerance Policy

HelixCode enforces a **zero-tolerance security policy** where: - **Critical vulnerabilities**: 0 acceptable - **High-severity vulnerabilities**: 0 acceptable - **Security hotspots**: Must be addressed - **Code quality issues**: Must be fixed before production

Defense in Depth

Security is implemented at multiple layers: 1. **Network Layer**: TLS, rate limiting, firewall rules 2. **Application Layer**: Input validation, authentication, authorization 3. **Data Layer**: Encryption, access controls, audit logging 4. **Infrastructure Layer**: Container security, orchestration policies

Authentication & Authorization

JWT-Based Authentication

HelixCode uses JSON Web Tokens (JWT) for stateless authentication:

```
auth:
  jwt_secret: "${HELIX_AUTH_JWT_SECRET}"  # 32+ byte secret
        required
  token_expiry: 24h
  session_expiry: 604800  # 7 days
  bcrypt_cost: 12
```

Security Requirements: - JWT secrets must be at least 32 bytes - Secrets stored in environment variables only - Tokens expire within 24 hours - Bcrypt cost factor of 12 for password hashing

Session Management

```
type Session struct {
    ID        string
    UserID    string
    Token     string
```

```
    ExpiresAt time.Time
    CreatedAt time.Time
}
```

Features: - Automatic session cleanup - Concurrent session limits per user - Session invalidation on logout - Redis-backed session storage

Role-Based Access Control (RBAC)

```
type Claims struct {
    UserID   string   `json:"user_id"`
    Username string   `json:"username"`
    Roles    []string `json:"roles"`
    jwt.RegisteredClaims
}
```

Built-in Roles: - `admin`: Full system access - `developer`: Code execution and project management - `user`: Basic platform access - `worker`: Distributed task execution

Input Validation & Sanitization

Security Manager

The `SecurityManager` provides comprehensive input validation:

```
type SecurityManager struct {
    logger         *logging.Logger
    scanResults    map[string]*FeatureScanResult
    securityScore  int
    criticalIssues int
    highIssues     int
    mutex          sync.RWMutex
}
```

Input Validation Rules

```
// Email validation
err := manager.ValidateEmail(email)

// URL validation
err := manager.ValidateURL(url)

// UUID validation
err := manager.ValidateUUID(id)

// JSON validation
err := manager.ValidateJSON(data)

// Custom validation rules
```

```
rule := &ValidationRule{
    Pattern:   `^[a-z][a-z0-9-]{2,30}$`,
    Message:   "Invalid project name format",
    MaxLength: 31,
}
```

Path Traversal Prevention

```
// Validate file path (prevent traversal)
err := manager.ValidatePath(path, basePath)
if err != nil {
    // Path contains traversal attempt
}

// Safe path joining
safePath := manager.SafeJoinPath(basePath, userPath)
```

Command Injection Prevention

```
// Validate shell command
err := manager.ValidateCommand(cmd, &CommandRules{
    AllowedCommands: []string{"git", "go", "npm"},
    BlockedPatterns: []string{";", "&&", "||", "|", "`"},
})
```

Input Sanitization

```
// Sanitize HTML
clean := manager.SanitizeHTML(input)

// Sanitize for SQL (use prepared statements instead)
clean := manager.SanitizeSQL(input)

// Sanitize for shell
clean := manager.SanitizeShell(input)
```

Network Security

TLS Configuration

Production Requirements: - TLS 1.3 minimum - Strong cipher suites only - Certificate pinning for APIs - HSTS headers enabled

Security Headers

Applied to all HTTP responses:

```
// Security headers applied:
// - Content-Security-Policy
```

```
// - X-Content-Type-Options: nosniff
// - X-Frame-Options: DENY
// - X-XSS-Protection: 1; mode=block
// - Strict-Transport-Security: max-age=31536000
// - X-Permitted-Cross-Domain-Policies: none
```

Rate Limiting

```
limiter := security.NewRateLimiter(&RateLimitConfig{
    RequestsPerMinute: 60,
    BurstSize:        10,
})
```

Rate Limits: - API endpoints: 60 requests/minute - Authentication: 5 attempts/minute - File uploads: 10 MB/hour - LLM requests: Provider-specific limits

CORS Configuration

```
server:
  cors:
    allowed_origins:
      - "https://app.helixcode.dev"
      - "https://*.helixcode.dev"
    allowed_methods: ["GET", "POST", "PUT", "DELETE"]
    allowed_headers: ["Authorization", "Content-Type"]
    allow_credentials: true
    max_age: 86400
```

API Key Management

Secure Key Generation

```
// Generate API key
key, err := manager.GenerateAPIKey()

// Hash API key for storage
hash := manager.HashAPIKey(key)

// Verify API key
valid := manager.VerifyAPIKey(key, hash)
```

Key Security: - 32-byte cryptographically secure keys - Argon2id hashing for storage - Automatic key rotation - Audit logging of key usage

Provider-Specific Keys

Required Environment Variables:

```
# Authentication
HELIX_AUTH_JWT_SECRET=your-super-secure-jwt-secret

# Database
HELIX_DATABASE_PASSWORD=your-secure-database-password

# Redis
HELIX_REDIS_PASSWORD=your-secure-redis-password

# LLM Providers (premium)
ANTHROPIC_API_KEY=sk-ant-your-key
OPENAI_API_KEY=sk-your-openai-key
GEMINI_API_KEY=your-gemini-key

# Free providers (optional)
GITHUB_TOKEN=ghp_your_github_token
OPENROUTER_API_KEY=sk-or-your-key
XAI_API_KEY=xai-your-key
```

Automated Security Scanning

Security Infrastructure

HelixCode includes comprehensive security scanning:

```
# Run complete security scan
docker-compose -f scripts/security_scan/docker-
        compose.security.yml up

# Execute security scan
docker exec helixcode-security-scanner /security-scripts/scan-
        all.sh
```

SonarQube Integration

Code Quality Analysis: - Security hotspots detection - Code coverage analysis - Technical debt assessment - Vulnerability scanning

Configuration:

```
sonarqube:
  host: "http://sonarqube:9000"
  project_key: "helixcode"
  exclusions: "**/vendor/**,**/test/**,**/mock/**"
```

Snyk Integration

Vulnerability Scanning: - Dependency vulnerability detection - License compliance checking - Container security scanning - Code vulnerability analysis

Zero-Tolerance Enforcement: - Critical vulnerabilities: 0 allowed - High vulnerabilities: 0 allowed - Medium/low: Must be reviewed

Security Scan Results

```
{
  "timestamp": "2024-01-01T00:00:00Z",
  "project": "HelixCode",
  "version": "1.0.0",
  "sonarqube": {
    "status": "scanned",
    "security_hotspots": 0,
    "code_quality_score": 95
  },
  "snyk": {
    "vulnerabilities": 0,
    "critical": 0,
    "high": 0,
    "status": "scanned"
  }
}
```

Container Security

Docker Security Best Practices

Security Scanner Dockerfile:

```
FROM golang:1.21-alpine AS builder
# Multi-stage build with minimal attack surface

FROM alpine:3.18
# Production stage with runtime dependencies only

RUN addgroup -S security && adduser -S -G security security
USER security
# Non-root user execution
```

Security Features: - Non-root user execution - Minimal base images - No unnecessary packages - Read-only filesystems where possible

Container Orchestration

Kubernetes Security:

```
apiVersion: v1
kind: Pod
metadata:
  name: helixcode-secure
```

```yaml
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1001
    fsGroup: 2000
  containers:
  - name: helixcode
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

Data Protection

Database Security

PostgreSQL Configuration:

```yaml
database:
  sslmode: "require"  # TLS required
  host: "postgres"
  user: "helix"
  password: "${HELIX_DATABASE_PASSWORD}"
```

Security Features: - TLS encryption in transit - Row-level security (RLS) - Prepared statements only - Connection pooling limits

Encryption at Rest

Data Encryption: - Database: Transparent Data Encryption (TDE) - Files: AES-256 encryption - Backups: Encrypted storage - Secrets: HashiCorp Vault integration

Audit Logging

Comprehensive Audit Trail:

```go
type AuditEvent struct {
    ID          string
    UserID      string
    Action      string
    Resource    string
    Timestamp   time.Time
    IPAddress   string
    UserAgent   string
    Success     bool
    Details     map[string]interface{}
}
```

Logged Events: - Authentication attempts - API key usage - File operations - LLM requests - Administrative actions

LLM Provider Security

Provider Authentication

Secure API Key Management: - Keys stored in environment variables - Never logged or exposed in responses - Automatic key rotation - Provider-specific rate limiting

Request Security

LLM Request Validation: - Input sanitization before LLM processing - Output filtering for sensitive data - Request/response logging (without keys) - Provider failover security

Free vs Premium Providers

Free Providers (No Keys Required): - XAI (Grok): up to 1M context per xAI's official model docs (the prior "2M" figure was stale), fast inference - OpenRouter: Multiple models, free tiers - GitHub Copilot: With subscription - Qwen: 2K requests/day

Premium Providers (API Keys Required): - Anthropic Claude: 200K context, advanced reasoning - Google Gemini: 2M context, multimodal - OpenAI: GPT-4, enterprise features - Azure OpenAI: Microsoft infrastructure

Worker Security

SSH-Based Worker Authentication

Secure Worker Connection:

```
workers:
  health_check_interval: 30
  health_ttl: 120
  max_concurrent_tasks: 10
  ssh_key_path: "/etc/helix/ssh_keys"
```

Security Features: - SSH key-based authentication only - No password authentication - Key rotation policies - Connection encryption

Worker Auto-Installation

Automated Secure Setup: - Automatic Helix CLI installation - Secure configuration deployment - Health monitoring and auto-recovery - Isolated execution environments

Monitoring & Alerting

Security Metrics

Real-time Monitoring:

```
type SecurityMetrics struct {
    CriticalIssues    int
    HighIssues        int
    SecurityScore     int
    FailedLogins      int
    BlockedRequests   int
    ActiveSessions    int
}
```

Alert Configuration

Security Alerts: - Critical vulnerability detection - Authentication failures (brute force) - Unauthorized access attempts - Configuration changes - Security policy violations

Notification Channels:

```
notifications:
  slack:
    webhook_url: "${HELIX_SLACK_WEBHOOK_URL}"
  email:
    smtp_server: "smtp.gmail.com"
    username: "${HELIX_EMAIL_USERNAME}"
    password: "${HELIX_EMAIL_PASSWORD}"
  telegram:
    bot_token: "${HELIX_TELEGRAM_BOT_TOKEN}"
    chat_id: "${HELIX_TELEGRAM_CHAT_ID}"
```

Incident Response

Security Incident Process

1. **Detection**: Automated monitoring alerts
2. **Assessment**: Security team evaluation
3. **Containment**: Isolate affected systems
4. **Eradication**: Remove security threats
5. **Recovery**: Restore normal operations
6. **Lessons Learned**: Update security measures

Breach Notification

Regulatory Compliance: - GDPR: 72-hour breach notification - CCPA: 45-day breach notification - Industry-specific requirements

Compliance & Standards

Security Standards

Implemented Standards: - OWASP Top 10 protection - NIST Cybersecurity Framework - ISO 27001 controls - SOC 2 Type II requirements

Compliance Features

Audit Controls: - Complete audit trails - Access logging - Change management - Security assessments

Development Security

Secure Development Practices

Code Review Requirements: - Security review for all code changes - Automated security scanning in CI/CD - Dependency vulnerability checks - Static application security testing (SAST)

CI/CD Security

Security Gates:

```
# .github/workflows/security.yml
- name: Security Scan
  run: |
    make security-scan
    if [ $? -ne 0 ]; then
      echo "Security scan failed - blocking deployment"
      exit 1
    fi
```

Production Deployment Security

Infrastructure Security

Production Requirements: - Network segmentation - Zero-trust architecture - Multi-factor authentication - Regular security assessments

Backup Security

Secure Backups: - Encrypted backup storage - Access controls on backups - Backup integrity verification - Disaster recovery testing

Security Testing

Automated Security Tests

```
# Run security test suite
make security-test

# Run vulnerability scans
make vuln-scan

# Run penetration tests
make pen-test
```

Security Test Categories

- **Unit Tests**: Security function validation
- **Integration Tests**: Authentication flows
- **E2E Tests**: Complete security scenarios
- **Penetration Tests**: External attack simulation

Emergency Procedures

Security Emergency Contacts

Emergency Response Team: - Security Lead: security@helixcode.dev - DevOps Lead: devops@helixcode.dev - Legal/Compliance: legal@helixcode.dev

Emergency Runbooks

Available Runbooks: - Data breach response - DDoS mitigation - Ransomware response - System compromise recovery

Security Updates & Maintenance

Regular Security Activities

Monthly: - Security patch deployment - Vulnerability scans - Access review - Security training

Quarterly: - Penetration testing - Security assessments - Compliance audits - Incident response drills

Annually: - Security architecture review - Third-party risk assessments - Business continuity testing

Conclusion

HelixCode's comprehensive security implementation ensures enterprise-grade protection for distributed AI development. The zero-tolerance policy, defense-in-depth approach, and automated security scanning provide robust protection against modern threats while maintaining development productivity.

Key Security Strengths: - Zero-tolerance vulnerability policy - Comprehensive automated scanning - Defense-in-depth architecture - Enterprise authentication and authorization - Secure container and orchestration practices - Complete audit and compliance capabilities

For additional security information, see the API Reference and Deployment Guide.

Sources verified

Sources verified 2026-05-29: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html ; https://docs.x.ai/docs/models ; https://docs.podman.io/en/latest/markdown/podman-compose.1.html — Confirmed against official sources on 2026-05-29: (1) OWASP Password Storage Cheat Sheet recommends **Argon2id** (m=19 MiB, t=2, p=1) as the primary password-hashing algorithm and **bcrypt work factor ≥10** for legacy systems — this

doc's `bcrypt_cost: 12` and Argon2id API-key hashing both meet current OWASP guidance. (2) xAI Grok context window is **up to 1M tokens** per the official model docs — the prior "2M context" claim in the Free Providers section was stale and has been corrected.

Negative findings / caveats (§11.4.99(B)): - **Tool-vendor pages not re-fetched this session** — SonarQube and Snyk are referenced only as integration names with config snippets (no version-pinned install instructions), so no version claim required re-verification. If a future revision pins SonarQube/Snyk versions or quotes their CLI flags, those MUST be re-verified against `https://docs.sonarsource.com/` and `https://docs.snyk.io/` per §11.4.99. - **Rule-1 (No CI/CD) inconsistency, not a doc-staleness issue:** §"CI/CD Security" still shows a `.github/workflows/security.yml` example, which contradicts HelixCode Rule 1 (No CI/CD Pipelines). Flagged here for a follow-up edit; out of scope for the §11.4.99 latest-source pass (this is an internal-policy contradiction, not an external-docs-staleness defect). - **xAI prompt caching unaddressed** in xAI's official docs (absence of documentation, not a confirmed capability either way). docs/COMPLETE_SECURITY_GUIDE.md